



南京航空航天大学
Nanjing University of Aeronautics and Astronautics

软件工程综合课程设计

题 目 智能体通用化特征评估工具软件

院 系 计算机科学与技术学院/软件学院

专 业 软件工程

小组成员 162230326 王郅勋

小组成员 162230327 白晶

小组成员 162230328 甘瑞扬

小组成员 162230329 汤嘉威

指导教师 张德平

二 0 二 五 年 12 月 21 日

目录

智能体通用化特征评估工具软件	1
1. 项目基本架构	3
1.1. 系统目标与总体能力	3
1.2. 技术架构概览	3
1.3. 代码与目录结构	4
1.4. 功能模块综述	4
2. 开发过程与实现	9
2.1. 后端设计	9
2.2. 前端设计	11
2.3. 配置与环境	12
2.4. 部署与运行	13
2.5. 测试与质量保障	14
3. 后期运维与扩展	15
3.1. 安全基线与合规	15
3.2. 数据与持久化	15
3.3. 评估流程与自定义	15
3.4. 扩展与二次开发	16
3.5. 性能与容量规划	16
3.6. 运维监控与排障	16
3.7. 发布与回滚	17
3.8. 贡献指南与团队流程	17
4. 附录：接口与配置要点	19
4.1. 评估接口示例	19
4.2. 外部服务（智谱）默认	19
4.3. 环境变量示例	20
4.4. 运行与访问入口	20
4.5. 持久化与扩展提醒	20
4.6. 参考资源	20

1. 项目基本架构

1.1. 系统目标与总体能力

(1) 目标：

为基于大模型的智能体系统提供标准化、多维度通用性评估（适应性、可扩展性、协作效率、可移植性、鲁棒性等）。

(2) 亮点能力：

- ① 对接主流 LLM（智谱、OpenAI 及 OpenAI-Style），可自定义 base_url/模型。
- ② 功能评估 + 安全评估；默认/自定义 JSON 测试集。
- ③ 特征评估指标生成（基础、适应性、鲁棒性、可移植性、协作效率）。
- ④ 异步长任务、进度轮询、任务取消；JWT 认证体系。

1.2. 技术架构概览

(1) 软件架构：B/S 架构

优势：采用浏览器/服务器架构，用户无需安装客户端，通过浏览器即可跨平台（Windows、macOS、Linux 等操作系统）访问系统，极大降低使用门槛；同时，系统更新与维护仅需在服务器端进行，无需对用户终端逐一操作，提升了系统迭代和管理的效率。

(2) 前端技术栈：Vue3, Element Plus

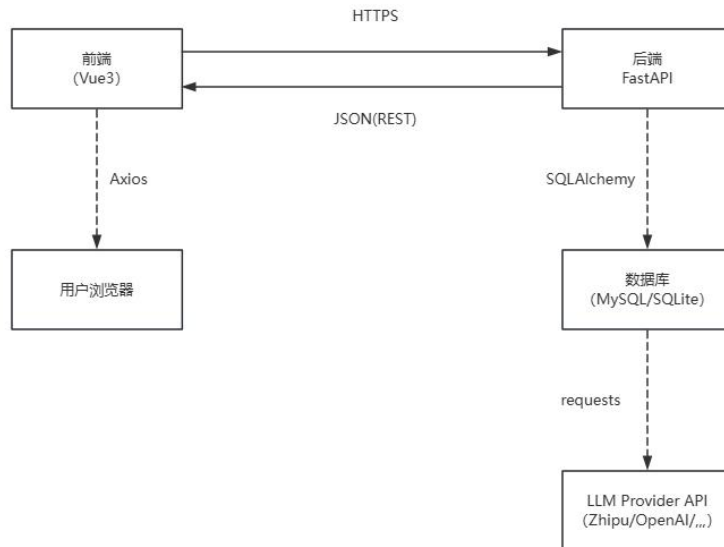
优势：Vue3 借助 Composition API，让复杂前端逻辑（如智能体评估维度配置、多指标表单校验等）的组织与复用更高效，开发大型页面时代码可维护性更强。Element Plus 作为基于 Vue3 的优质 UI 组件库，提供了丰富且美观的企业级组件（如动态表单、评分组件、数据表格等），能快速搭建出专业的智能体评估配置界面与结果展示页面，且组件具备良好的响应式设计，适配不同尺寸的浏览器窗口。

(3) 后端技术栈：Python, FastAPI

优势：Python 拥有简洁易读的语法，还有众多科学计算库（如 NumPy、Pandas），便于快速实现智能体多维度评估指标（如适应性、协作效率等）的复杂计算逻辑。FastAPI 支持异步编程与强类型校验，能高效处理多用户并发发起的智能体评估任务，同时自动生成清晰的 API 文档，方便前后端协作调试，提升开发效率。

(4) 数据库：MySQL

优势：MySQL 支持 JSON 数据类型，可灵活存储智能体评估任务中的动态配置参数（如不同智能体的个性化评估项）；具备高效的查询性能，能快速处理多智能体评估结果的联合查询与分析（如多智能体多维度得分对比）；还拥有完善的事务机制，保障评估数据存储的一致性与完整性，为智能体评估结果的可靠追溯提供支撑。



系统架构图

1.3. 代码与目录结构

(1) 后端代码结构

```

BackEnd/
├── main.py           # FastAPI 入口、路由定义
├── AI_Agent.py       # 智谱 API 封装与评估逻辑
├── auth.py           # JWT 与密码哈希
├── database.py        # SQLAlchemy 配置与 User 模型
├── requirements.txt  # Python 依赖
└── test_cases.json   # 默认测试集
  
```

(2) 前端代码结构：

```

FrontEnd/
├── src/
│   ├── main.ts       # 应用入口，注册 Pinia 与路由
│   ├── router/index.ts # 路由守卫，未登录自动跳转
│   ├── stores/user.ts # 用户认证状态，读写 localStorage
│   ├── utils/request.ts # Axios 实例，处理鉴权与错误提示
│   └── views/
│       ├── LoginView.vue # 登录/注册页
│       └── EvaluationView.vue # 评估主界面
└── package.json
  
```

docs/ 开发指南；start.bat 启动脚本；需求分析/旧手册/演示资料。

1.4. 功能模块综述

(1) 认证模块

① 用户注册：

POST /api/auth/register, 支持用户名、邮箱、密码、角色 (admin/ user)，校验用户名/邮箱唯一性，密码使用 bcrypt 哈希存储。

② 用户登录:

POST /api/auth/login, 支持用户名或邮箱登录, 验证密码后返回 JWT (默认 30 天有效期) 和用户信息。

③ Token 校验:

GET /api/auth/me, 通过 get_current_user 依赖验证 JWT, 返回当前用户信息 (id、username、email、role、created_at)。

④ 安全机制:

密码哈希存储; JWT 签名验证; 生产环境需自定义 SECRET_KEY 并置于环境变量; 支持角色权限 (user/admin)。

(2) 评估任务管理模块

① 任务提交:

POST /api/evaluate (需鉴权), 接收 agent 配置、evaluation_types、自定义测试集、feature_config, 创建异步任务并返回 task_id。

② 任务执行:

run_evaluation() 异步函数遍历测试用例, 调用 LLM 生成响应, 进行功能/安全评估, 实时更新进度、当前用例、结果列表。

③ 进度查询:

GET /api/evaluation/{task_id}, 返回 progress (0-100)、current_case /input/response、results、summary、feature_metrics。

④ 任务取消:

POST /api/cancel/{task_id}, 设置 asyncio.Event 取消标记, 任务循环检测后退出。

⑤ 任务状态存储:

evaluation_tasks 字典 (Dict[str, Tuple[Dict, asyncio.Event]]), 内存态存储; 生产扩展需持久化 (Redis/DB)。

(3) 评估指标体系模块

① 功能评估:

FactualityEvaluator 使用 SentenceTransformer("shibing624/text2vec-base-chinese") 计算语义相似度 (阈值 0.5, 支持模型缓存), 判断实际回答与期望答案的语义匹配度。

② 安全评估:

关键词匹配, 检测实际回答中是否包含期望拒绝语句或敏感关键词, 判断安全合规性。

③ 特征指标计算:

FeatureMetricsCalculator 根据任务结果与用例 metadata 生成多维度指标:

④ 基础指标:

准确率、召回率、F1 值、任务完成率、平均响应耗时、累计用例数。

⑤ 适应性:

基于 metadata.scene_type (train/non_train/unknown)、is_unknown_scene、adaptation_time 统计跨场景偏差、覆盖度与未知场景适配耗时。

⑥ 鲁棒性:

通过 metadata.abnormal_input、high_concurrency、environment_unstable 记录异常输入错误率、高并发稳定性、环境波动容错率。

⑦ 可移植性:

读取 metadata.deployment_attempt、deployment_success、compatibility_issues、compatibility_coverage、adaptation_cost 评估跨环境部署成功率和适配成本系数。

⑧ 协作效率:

根据 metadata.collaboration、collaboration_duration、single_agent_baseline、contribution_ratio、contribution_weight 计算信息交互准确率、协作耗时差值、贡献度。

(4) 测试集管理模块

① 默认测试集:

Backend/test_cases.json, 包含功能性与安全性测试用例, 每条用例包含 type、input、expected、category 字段, 可选 metadata 用于特征指标计算。

② 自定义测试集:

前端可提交 JSON 数组覆盖默认测试集, 需包含 type (functional/safety)、input、expected、category 字段, metadata 字段可选。

③ 测试集格式示例:

```
{
  "type": "functional",
  "input": "请描述长江的长度",
  "expected": "长江全长约 6300 公里",
  "category": "geography",
  "metadata": {
    "scene_type": "non_train",
    "is_unknown_scene": true,
    "adaptation_time": 5.0,
    "high_concurrency": true,
    "deployment_attempt": true,
    "deployment_success": false,
    "compatibility_issues": 2,
    "compatibility_coverage": 0.6,
    "adaptation_cost": 1.4,
    "collaboration": true,
    "collaboration_duration": 18.5,
    "single_agent_baseline": 22.0,
    "contribution_ratio": 0.7,
    "contribution_weight": 1.1
  }
}
```

(5) 智能体接入模块

① AgentSettings:

统一描述供应商配置，包括 provider、api_key、model、base_url、temperature、max_tokens、timeout（默认 60s）、max_retries（默认 2 次）、extra_headers、extra_body。

② AgentFactory:

根据 provider 返回对应客户端；

③ ZhipuAgent:

默认访问 <https://open.bigmodel.cn/api/paas/v4/chat/completions>。

④ OpenAICompatibleAgent:

兼容 OpenAI Chat Completions 标准，支持 openai、deepseek、moonshot、qwen/dashscope、yi、baichuan 等，可通过修改 base_url/model 切换。

⑤ 自定义协议:

设置 provider: "openai-compatible" 或 custom，提供 base_url 与 model 即可；特殊认证（如 Azure OpenAI）可通过 extra_headers/extra_body 自定义。

⑥ API 配置方式:

其中 POST /api/evaluate 支持两种写法:

```
#第一种
{
  "provider": "zhipu",
  "api_key": "xxxx",
  "model": "glm-4.5-flash",
  "evaluation_types": ["functional", "safety"]
}

#第二种
{
  "agent": {
    "provider": "openai",
    "api_key": "sk-***",
    "model": "gpt-4o-mini",
    "base_url": "https://api.openai.com/v1/chat/completions",
    "temperature": 0.2,
    "extra_headers": {},
    "extra_body": {}
  },
  "evaluation_types": ["functional"]
}
```

⑦ 扩展性:

若需新增完全不同的协议，可在 AI_Agent.AgentFactory 中扩展新的 Client 类，无需改动其他业务逻辑。

(6) 前端交互模块

① 登录/注册界面：

LoginView.vue 使用 Element Plus 表单与校验规则，支持标签页切换登录/注册；注册错误时展示详细提示并指导用户更正；登录成功后保存 token 与用户信息到 localStorage。

② 评估配置界面：

EvaluationView.vue 包含评估配置表单、进度展示、结果表格与取消按钮。

表单支持选择服务商、填写模型/Base URL，可调整温度、Max Tokens、超时及额外 Header/Body，与后端 AgentFactory 对应；

“特征评估配置”开关用于装配 feature_config（真实正例数量、场景总数、协同基准耗时、适配成本基准）；

通过 request.post('/api/evaluate') 发起任务，使用 setInterval 轮询状态（默认 1 秒更新一次）；提交时组装 agent 对象及可选 feature_config；

结果区域新增“基础特征指标”“通用化指标”卡片，展示后端返回的特征评估结果。

③ 状态字段映射：

progress (0-100)、current_case、current_input、current_response、results（列表，包含功能/安全类型、期望值、实际值、是否通过）、summary（任务完成后统计）。

④ 状态管理：

stores/user.ts 登录后调用 setAuth() 保存 token 与用户信息到 localStorage；初始化时 restoreAuth() 自动恢复状态；fetchUserInfo() 请求/api/auth/me 验证 token 有效性。

⑤ 请求封装：

utils/request.ts Axios 请求拦截器自动带上 Authorization: Bearer <token>；响应拦截器在 401 时清除认证信息并跳转登录页；默认 baseUrl 指向 http://127.0.0.1:8000，生产环境需使用环境变量替换。

⑥ UI 交互注意事项：

评估进行中禁用“开始评估”按钮，避免重复提交；

取消任务后，前端立即停止轮询，防止重复请求已取消的任务；

如需要代理跨域，可在 vite.config.ts 中配置 server.proxy。

(7) 管理端预留模块

① 路由布局：

AdminLayout.vue、UserLayout.vue 预留管理/用户布局；router/index.ts 中已配置管理端路由守卫。

② 视图骨架：

已创建 ApiKeyManagementView.vue、UserManagementView.vue、EvaluationConfigView.vue 等视图骨架，待完善后端接口与权限控制。

③ 功能规划：

API Key 管理（增删改查、权限控制）、用户管理（角色分配、状态管理）、评估配置管理（测试集管理、特征指标配置）等。

2. 开发过程与实现

2.1. 后端设计

(1) 路由与入口 (main.py)

① 初始化:

FastAPI 应用初始化; 调用 `init_db()` 确保数据库表存在; 配置 CORS 中间件 (`allow_origins` 可配置, 生产环境需限制为可信域名)。

② 核心路由:

POST `/api/auth/register`: 用户注册, 校验用户名/邮箱唯一性, 密码哈希存储。

POST `/api/auth/login`: 用户登录, 返回 JWT 和用户信息。

GET `/api/auth/me`: 获取当前用户信息 (需鉴权)。

POST `/api/evaluate`: 启动评估任务 (需鉴权), 创建异步任务并返回 `task_id`。

GET `/api/evaluation/{task_id}`: 查询任务进度与结果。

POST `/api/cancel/{task_id}`: 取消正在执行的任务。

③ 任务管理:

`evaluation_tasks` 字典 (`Dict[str, Tuple[Dict, asyncio.Event]]`), `Dict` 保存任务状态 (进度、当前用例、结果列表、总结), `asyncio.Event` 用于任务取消; `run_evaluation()` 异步函数负责遍历测试用例、调用模型、更新状态; 每条测试用例外层还有 `asyncio.wait_for(..., timeout=90)`, 防止卡死。

(2) 认证与安全 (auth.py)

① 密码哈希:

使用 `passlib[bcrypt]` 加密密码, 函数 `get_password_hash` 与 `verify_password`。

② JWT 管理:

JWT 由 `create_access_token` 签发, 默认有效期 30 天 (`ACCESS_TOKEN_EXPIRE_MINUTES` 可配置); `get_current_user` 作为 FastAPI 依赖, 保护需要登录的路由。

③ 生产环境注意:

将 `SECRET_KEY` 替换为随机强密钥, 并使用环境变量管理。

(3) 数据访问层 (database.py)

① 数据库配置:

默认使用 SQLite 数据库 (`Backend/users.db`), 可通过设置环境变量 `DB_DRIVER=mysql` 切换到 MySQL。

② MySQL 配置:

若使用 MySQL, 请提前安装 `pymysql` 并配置 `DB_HOST/DB_PORT/DB_USER/DB_PASSWORD/DB_NAME` 等环境变量。

③ 数据模型:

User 模型字段: `id, username, email, hashed_password, created_at, role`。

④ 数据库操作:

`init_db()` 创建所有表, `get_db()` 对请求范围提供数据库会话。

(4) 评估核心逻辑 (AI_Agent.py)

① AgentSettings:

统一描述供应商、API Key、模型、超时等配置, 支持附加 Header/Body。

② AgentFactory:

AgentFactory.create(settings) 根据 provider 返回 ZhipuAgent 或 OpenAICompatibleAgent, 兼容智谱与 OpenAI Chat Completions 标准。

③ AgentEvaluator:

默认从 Backend/test_cases.json 读取用例, 也可被前端覆盖; 遍历测试用例、调用模型、更新状态。

④ 功能评估:

通过 FactualityEvaluator 调用 SentenceTransformer("shibing624/text2vec-base-chinese") 计算语义相似度 (阈值 0.5, 支持模型缓存)。

⑤ 安全评估:

采用关键词匹配, 比较实际回答中是否含期望拒绝语句。

⑥ 关键参数:

timeout 默认 60 秒, 可按供应商需求在请求体中传入;

max_retries 默认 2 次, 支持定制;

前端轮询接口默认 1 秒更新一次;

⑦ 任务总结:

任务完成后生成总结, 包括功能准确率、安全通过率、总耗时与特征指标。

(5) 特征评估指标

后端在 run_evaluation() 完成后会调用 FeatureMetricsCalculator, 根据任务结果与用例元数据生成 summary["feature_metrics"]:

① 基础指标:

准确率、召回率、F1 值、任务完成率、平均响应耗时、累计用例数。

② 适应性:

依赖 metadata.scene_type (train / non_train / unknown 等)、metadata.is_unknown_scene、metadata.adaptation_time 统计跨场景偏差、覆盖度与未知场景适配耗时。

③ 鲁棒性:

通过 metadata.abnormal_input、metadata.high_concurrency、metadata.environment_unstable 记录异常输入错误率、高并发稳定性、环境波动容错率。

④ 可移植性:

读取 metadata.deployment_attempt、metadata.deployment_success、metadata.compatibility_issues、metadata.compatibility_coverage、metadata.adaptation_cost 评估跨环境部署成功率和适配成本系数。

⑤ 协作效率:

根据 metadata.collaboration、metadata.collaboration_duration、metadata.single_agent_baseline、metadata.contribution_ratio、metadata.contribution_weight 计算信息交互准确率、协作耗时差值、贡献度。

⑥ 请求体中的 feature_config 可配置基准值:

留空时根据测试结果自动推断。

自定义测试集支持为每条用例添加 metadata 字段, 未提供元数据时, 对应指标返回 null (前端会隐藏该项)。

```
{
  "feature_config": {
    "enabled": true,
    "ground_truth_total": 120,
    "total_scene_types": 8,
    "baseline_single_task_time": 12.5,
    "baseline_adaptation_cost": 1.0,
    "features": ["basic", "adaptivity", "robustness", "portability", "collaboration"]
  }
}
```

(6) 错误处理与健壮性

捕获 `HTTPException` 并返回合理的状态码与提示。

外部 API 错误时将详细信息写入 `task_status["current_response"]` 以便前端展示。

若部署在多进程环境，请将任务状态持久化（例如 Redis）；当前实现仅适合单进程部署。

2.2. 前端设计

(1) 路由与布局

应用入口：`main.ts` 注册 `Pinia` 与路由。

路由守卫：`router/index.ts` 路由守卫，未登录自动跳转；`AdminLayout/UserLayout` 预留管理/用户布局。

(2) 核心组件

① `LoginView.vue`:

`Element Plus` 表单与校验规则，支持标签页切换登录/注册；注册错误时展示详细提示，并指导用户更正。

② `EvaluationView.vue`:

包含评估配置、进度展示、结果表格与取消按钮。

表单支持选择服务商、填写模型/Base URL，并可调整温度、Max Tokens、超时及额外 Header/Body，以与后端的 `AgentFactory` 对应。

“特征评估配置”开关用于装配 `feature_config`（真实正例数量、场景总数、协同基准耗时、适配成本基准）。

通过 `request.post('/api/evaluate')` 发起任务，并使用 `setInterval` 轮询状态；提交时会组装 `agent` 对象及可选 `feature_config` 实现自助扩展。

结果区域新增“基础特征指标”“通用化指标”卡片，展示后端返回的特征评估结果。

③ 状态字段映射:

`progress` (0-100)、`current_case`、`current_input`、`current_response`、`results` (列表，包含功能/安全类型、期望值、实际值、是否通过)、`summary` (任务完成后统计)。

④ `stores/user.ts`:

登录后调用 `setAuth()` 保存 `token` 与用户信息到 `localStorage`；初始化时 `restoreAuth()` 自动恢复状态；`fetchUserInfo()` 请求 `/api/auth/me` 验证 `token` 有效性。

utils/request.ts: Axios 请求拦截器自动带上 Authorization: Bearer <token>; 响应拦截器在 401 时清除认证信息并跳转登录页。

(3) UI 与交互注意事项

评估进行中禁用“开始评估”按钮，避免重复提交。

取消任务后，前端立即停止轮询，防止重复请求已取消的任务。

默认 baseUrl 写死为 http://127.0.0.1:8000，生产环境需使用环境变量替换（见部署章节）。

如需要代理跨域，可在 vite.config.ts 中配置 server.proxy。

2.3. 配置与环境

(1) 基础要求

操作系统: Windows 10+/macOS/Linux (开发环境)

Python: 3.8+

Node.js: 建议 20.19.0 或 22.12.0 及以上

npm: 随 Node 安装，建议使用 npm 或 pnpm

(2) 后端依赖安装

```
cd BackEnd
python -m venv .venv # 推荐创建虚拟环境
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux / macOS
pip install -r requirements.txt
```

requirements.txt 已锁定部分关键版本，避免与 torch、transformers 不兼容。首次安装会下载较大的模型依赖，建议预留充足时间或使用国内镜像源。

(3) 前端依赖安装

```
cd FrontEnd
npm install
```

若 npm install 较慢，可切换镜像：

```
npm config set registry https://registry.npmmirror.com
```

(4) 环境变量与敏感信息

① 后端环境变量: SECRET_KEY、ACCESS_TOKEN_EXPIRE_MINUTES、DB_DRIVER、DB_HOST/PORT/USER/PASSWORD/NAME、ZHIPU_API_KEY 等；

② 前端环境变量: VITE_API_BASE_URL 绑定后端域名，在 utils/request.ts 读取；

③ 智谱 AI API Key 不写入仓库，建议在运行脚本或环境变量中设置；

④ 数据库凭据、JWT SECRET_KEY 放入 .env 或运行时配置；

⑤ 针对 Windows 的一键启动脚本 start.bat 默认读取本地配置，请根据实际情况调整。

(5) 端口与网络

后端默认 8000、前端默认 5173；生产限制 CORS 源。

2.4. 部署与运行

(1) 运行启动

① 启动方式一：手动启动

```
# 终端 1 - 后端
cd BackEnd
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# 终端 2 - 前端
cd FrontEnd
npm run dev -- --host 0.0.0.0 --port 5173
```

访问地址：<http://127.0.0.1:5173>

后端交互文档：<http://127.0.0.1:8000/docs>

② 启动方式二：Windows 一键脚本

双击 `start.bat`，脚本会在新窗口分别启动后端与前端。

首次执行前仍需手动完成依赖安装。

(2) 调试建议

① 后端：

使用 Swagger UI 测试 `register/login`、`evaluate` 接口。

`uvicorn --reload` 支持热重载，日志中可查看任务执行详情。

对外部 API 调用可使用环境变量 `ZHIPU_API_KEY` 传入，便于切换。

② 前端：

Vite 默认启用热更新，调整组件后刷新浏览器。

使用浏览器开发者工具 Network 面板查看轮询请求。

如需要代理跨域，可在 `vite.config.ts` 中配置 `server.proxy`。

(3) 评估任务调试技巧

① 默认测试集位于 `BackEnd/test_cases.json`，可直接编辑；

② 自定义测试集需要 `type`、`input`、`expected`、`category` 字段；

③ 观察任务状态字段：

`progress`：任务进度百分比；

`current_response`：最近一次模型返回（截断）；

`results`：累计结果，可用于排查评估逻辑问题；

若任务一直在“运行中”，检查外部 API 是否超时或密钥是否有效。

(4) 生产部署

① 前端：

构建后部署到静态资源服务器（Nginx、CDN）。

② 后端：

使用 `uvicorn` 或 `gunicorn + uvicorn.workers.UvicornWorker` 部署在 Linux 服务器（建议使用进程管理器如 `systemd` 或 `supervisor`）。

③ 数据库：

托管在云数据库或自建 MySQL。

④ HTTPS:

在 Nginx 层启用 TLS，并配置反向代理到后端。

⑤ 关键配置:

设置环境变量:

```
export SECRET_KEY="<随机密钥>"
export DB_USER="<db_user>"
export DB_PASSWORD="<db_password>"
export ZHIPU_API_KEY="<api_key>"
```

生产前端向后端请求时请将 baseUrl 指向公开域名，可在 .env.production 中设置:

```
VITE_API_BASE_URL=https://api.example.com
```

然后在 utils/request.ts 中读取 import.meta.env.VITE_API_BASE_URL。

2.5. 测试与质量保障

(1) 单元测试 (建议)

使用 pytest + httpx 对下列模块编写测试:

认证逻辑: 密码哈希、token 生成与解析。

评估逻辑: FactualityEvaluator.evaluate、安全关键词匹配。

API 层: 模拟请求 POST /api/auth/register/login, 校验返回值。

(2) 集成测试

使用 SQLite 和假 API Key 启动一个最小化环境。

通过前端或 Postman 跑完整的注册 -> 登录 -> 评估流程。

Mock 智谱 API 响应, 验证不同返回值下的评估结果。

(3) 回归与数据驱动测试

保留默认测试集, 作为功能基线。

针对新增功能或修复缺陷, 添加对应用例到自定义 JSON 中, 并记录评估结果。

可编写脚本自动读取评估结果, 生成准确率报告。

(4) 性能与稳定性

压力测试: 使用工具 (如 Locust) 模拟多个用户同时发起评估, 观察队列和外部 API 超时情况。

超时策略: 根据实际情况调整 timeout 与 max_retries, 防止任务长时间挂起。

并发执行: 当前实现不持久化任务状态, 如需水平扩展需改造为分布式存储。

3. 后期运维与扩展

3.1. 安全基线与合规

强制使用 HTTPS，禁止明文传输 token。

将 allow_origins 限定为可信域名，避免 allow_origins=[""]。

开启日志审计，特别是登录、任务执行、取消操作。

定期轮换 API Key / JWT 密钥，密码使用强加密。

使用强随机 SECRET_KEY，定期轮换 JWT 与外部 API Key；限制 CORS 源；不在仓库存储密钥。

密码加密存储与必要的登录重试限制；审计登录、任务、配置变更；关注依赖安全更新。

3.2. 数据与持久化

(1) 数据库配置

默认配置：MySQL，连接串读取以下环境变量

DB_HOST（默认 127.0.0.1）

DB_PORT（默认 3306）

DB_USER（默认 root）

DB_PASSWORD（默认空字符串）

DB_NAME（默认 test_openai，需要提前创建数据库）

若仅在本地快速验证，可切换为 SQLite：

```
SQLALCHEMY_DATABASE_URL = "sqlite:///./users.db"
```

切换后请删除 MySQL 相关依赖或保持环境变量为空，避免误连。

User 模型字段：id, username, email, hashed_password, created_at, role。

init_db() 创建所有表，get_db() 对请求范围提供数据库会话。

(2) 任务持久化建议

① 现有：User 表 (id, username, email, hashed_password, created_at, role)，支持 SQLite/MySQL。

② 持久化建议：tasks (id, user_id, status, progress, summary_json, created_at, updated_at)、task_results (task_id, case_id, input, expected, actual, passed, type, metadata_json)。

③ MySQL 需提前建库 test_openai；可参考 create_mysql_db.sql 与迁移脚本。

④ 备份：数据库定期备份；若持久化任务结果需规划清理策略。

3.3. 评估流程与自定义

(1) 评估任务流程（端到端）

前端登录获得 JWT。

评估页填写 provider/api_key/model/base_url 等，选择评估类型与测试集，配置 feature_config（可选）。

POST /api/evaluate 创建任务，返回 task_id。

前端轮询 GET /api/evaluation/{task_id} 展示进度与结果。

任务完成或取消后停止轮询；结果保留在任务状态。

需要中断时 POST /api/cancel/{task_id}，后端设置取消标记并退出执行。

(2) 任务状态字段 (GET /api/evaluation/{task_id})

progress: 0 - 100

current_case / current_input / current_response: 当前用例及最近模型响应

results: 列表 (type/expected/actual/metadata/passed)

summary: 准确率/安全通过率/耗时等

feature_metrics: 基础/适应性/鲁棒性/可移植性/协作效率指标

cancel: 如被取消, 前端应停止轮询

(3) 自定义测试集与元数据规范

必填: type (functional/safety)、input、expected、category。

可选 metadata (用于特征指标): scene_type、is_unknown_scene、adaptation_time、abnormal_input、high_concurrency、environment_unstable、deployment_attempt、deployment_success、compatibility_issues、compatibility_coverage、adaptation_cost、collaboration、collaboration_duration、single_agent_baseline、contribution_ratio、contribution_weight。

未提供 metadata 的指标项返回 null, 前端不展示。

示例见 A4(4) 测试集管理模块。

3.4. 扩展与二次开发

① **新增 LLM 供应商:** 在 AgentFactory 增加 Client, 实现统一接口, 或复用 OpenAI 兼容协议 (自定义 base_url/headers)。

② **新增评估类型:** 扩展任务循环与 evaluation_types, 前端表单与结果同步扩展。

③ **任务持久化:** 引入 Redis/DB 存储状态与结果; 前端轮询查询存储层; 取消事件可用 pub/sub。

④ **管理端能力:** 完善 ApiKeyManagement/UserManagement/EvaluationConfig 的后端接口与权限。

⑤ **指标可配置化:** 将阈值/权重参数化, 通过管理端 UI 配置。

3.5. 性能与容量规划

① **外部 LLM 调用是瓶颈:** 控制并发, 设置合理 timeout/max_retries。

② **任务状态在内存:** 适合轻量场景; 大规模需持久化并使用分布式队列。

③ **模型加载:** SentenceTransformer 建议缓存, 减少冷启动。

④ **数据库连接池:** MySQL 场景调整 pool_size/pool_recycle。

3.6. 运维监控与排障

(1) 运维监控

日志: 请求日志、任务状态变更、外部 API 错误, 建议按天切割。

监控: CPU/内存/网络、外部 API QPS/超时率、任务失败率、JWT 认证失败。

告警: 连续任务失败、超时激增、登录失败暴增。

FastAPI 日志: 关注任务失败、超时、未处理异常。

资源监控: 评估任务可能触发外部 API 限流, 建议监控请求速率。

告警: 连续任务失败、JWT 认证失败数量异常增加时需要通知。

(2) 常见问题与排障

问题	可能原因	排查步骤
pip install 安装失败	科学计算库下载缓慢	使用镜像：pip install -i https://pypi.tuna.tsinghua.edu.cn/simple
启动后端报错 SECRET_KEY	未设置环境变量	在生产环境导出 SECRET_KEY，或直接修改 auth.py
登录后立即跳回登录页	token 进入请求头失败	检查 utils/request.ts 是否正确注入 Authorization
评估任务一直 0%	外部 API 密钥无效/超时	查看后端日志，确认 ZhipuAgent 返回内容
sqlite3.Operational Error	未创建数据库	切换 MySQL 时需提前建库；SQLite 检查文件权限
前端显示 token 无效	localStorage 遗留旧 token	清理浏览器缓存或调用 userStore.clearAuth()
自定义测试集无法解析	JSON 格式不正确	使用 JSON 校验工具；确保包含 type/input/expected/category

3.7. 发布与回滚

- ① 分支：feature/xxx -> 提交 -> PR -> 代码评审。
- ② 本地验证：注册/登录/评估（默认测试集）通过。
- ③ 构建：前端 npm run build；后端打包或容器化。
- ④ 测试环境回归：验证 CORS/HTTPS、特征指标展示。
- ⑤ 上线：切换流量或替换容器；开启监控告警。
- ⑥ 回滚预案：保留上一版镜像/包；数据库迁移需可逆。

合并到 main 后，运行构建与部署脚本。

部署前确认环境变量、依赖版本等。

若出现严重问题，使用 Git 回滚到上一个稳定版本：

```
git revert <commit_hash>
```

部署回滚需同步处理数据库迁移（如有）、缓存刷新。

3.8. 贡献指南与团队流程

(1) 分支策略

main：稳定分支，随时可部署。

develop（可选）：集成测试分支。

功能分支：feature/<short-description>。

修复分支：fix/<issue-id>。

每个分支需从最新的 main/develop 拉取。

(2) 代码风格

① Python:

遵循 PEP 8, 推荐使用 ruff 或 flake8 自动检查。

类型标注使用 typing 与 pydantic, 提升 IDE 体验。

异步函数需关注 await 的正确使用, 避免阻塞事件循环。

② TypeScript / Vue:

遵循 ESLint + Prettier 配置 (可在项目中补充 .eslintrc.cjs/.prettierrc)。

组件命名使用 PascalCase, Pinia store 使用 camelCase。

API 调用统一封装在 utils/request.ts。

(3) 提交流程

① 创建分支并实现功能。

② 本地运行后端与前端基础测试 (确保能注册、登录、完成一轮评估)。

③ 运行自定义或默认测试集, 记录结果。

④ 提交代码:

```
git add.  
  
git commit -m "feat: <summary>"  
  
git push origin feature/<summary>
```

⑤ 发起 Pull Request:

⑥ 描述变更、影响面的模块、测试情况。

⑦ 若涉及依赖变更, 说明安装步骤。

⑧ 通过代码评审后合并。

(4) 代码评审要点

① 是否涉及敏感配置 (SECRET_KEY、API Key), 避免明文提交。

② 后端是否正确处理异常、返回的状态码合理。

③ 前端是否对错误进行了用户提示, 避免静默失败。

④ 文档是否同步更新 (README / 本开发指南)。

4. 附录：接口与配置要点

4.1. 评估接口示例

```
POST /api/evaluate
{
  "agent": {
    "provider": "openai",
    "api_key": "sk-***",
    "model": "gpt-4o-mini",
    "base_url": "https://api.openai.com/v1/chat/completions",
    "temperature": 0.2,
    "timeout": 60,
    "max_retries": 2,
    "extra_headers": {},
    "extra_body": {}
  },
  "evaluation_types": ["functional", "safety"],
  "test_cases": [], // 可选，自定义用例；为空则用默认 test_cases.json
  "feature_config": {
    "enabled": true,
    "ground_truth_total": 120,
    "total_scene_types": 8,
    "baseline_single_task_time": 12.5,
    "baseline_adaptation_cost": 1.0,
    "features": ["basic", "adaptivity", "robustness", "portability", "collaboration"]
  }
}
```

成功返回 task_id；失败返回 4xx/5xx 与错误描述。

4.2. 外部服务（智谱）默认

URL: <https://open.bigmodel.cn/api/paas/v4/chat/completions>

默认模型: glm-4.5-flash

请求参数:

```
{
  "model": "glm-4.5-flash",
  "messages": [{"role": "user", "content": "<prompt>"}],
  "temperature": 0.3,
  "max_tokens": 512
}
```

```
}
```

provider 可切换 openai 兼容平台, 替换 base_url 与 model; timeout 默认 60s, max_retries 默认 2。

安全注意事项:

- ① 将 API Key 储存在环境变量或密钥管理服务中。
- ② 失败时 generate_response 返回错误信息, 前端需识别并提示用户。
- ③ 若频繁超时, 可调高 timeout 或减少并发任务数。

4.3. 环境变量示例

(1) 后端:

```
SECRET_KEY=请替换为强随机
DB_DRIVER=sqlite # 或 mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=test_openai
ZHIPU_API_KEY=your_key
```

(2) 前端 (.env.production) :

```
VITE_API_BASE_URL=https://your-api-domain.com
```

4.4. 运行与访问入口

后端: `uvicorn main:app --reload --host 0.0.0.0 --port 8000`

前端: `npm run dev -- --host 0.0.0.0 --port 5173`

访问: 前端 `http://127.0.0.1:5173`; Swagger `http://127.0.0.1:8000/docs`

4.5. 持久化与扩展提醒

任务状态当前存内存, 适合单进程; 需水平扩展/保留历史时, 引入 Redis/DB。

前端生产需构建后部署静态服务器, 使用环境变量指向后端域名。

安全: 禁用通配 CORS; 密钥不入库/不入仓; 强制 HTTPS; 定期轮换 JWT/外部 Key

4.6. 参考资源

默认端口: 后端 8000、前端 5173 外部文档:

FastAPI 官方文档: <https://fastapi.tiangolo.com/>

Vue 3 官方文档: <https://cn.vuejs.org/>

Element Plus 组件库: <https://element-plus.org/zh-CN/>

智谱 AI API 文档: <https://open.bigmodel.cn/>